CSC 452 — Formal Models of Computation

Student Study Notes (with Explanations & Examples)

Course Overview

CMP 452 – Formal Models of Computation (3 Units) (L: 45)

Topics covered:

- **Automata Theory:** Roles of models in computation, Finite State Automata, Pushdown Automata, Formal Grammars, Parsing, Relative powers of formal models.
- **Basic Computability:** Turing Machines, Universal Turing Machines, Church's Thesis, Solvability and Decidability.

---

1. Introduction

What is the Theory of Computation (TOC)?

The **Theory of Computation (TOC)** is a branch of theoretical computer science that asks two fundamental questions:

1. **Can** a given problem be solved by a computer at all?
2. **How efficiently** can it be solved?

> Think of it this way: before you spend months building a program to solve a problem, TOC tells you whether a solution even *exists*, and whether it can run in reasonable time.

The field is divided into three major branches:

Branch	What it studies
Automata Theory	Abstract machines (models) and the problems they can solve
Computability Theory	What problems can or cannot be solved by any computer at all
Computational Complexity Theory	How *efficiently* solvable problems can be solved (time, memory)

To study computation rigorously, computer scientists use a **model of computation** — a mathematical abstraction of a computer. The most commonly examined model is the **Turing machine**.

Why not just use a real computer? Real computers change over time (hardware, OS, languages). A mathematical model is universal and timeless — its results hold regardless of the physical machine.

---

2. Why Study the Theory of Computation (TOC)?

i. The Shelf Life of Programming Tools

Programming tools and languages constantly change:

- Early computing used **Machine Code** and **Assembly**
- In **1957**, **Fortran** introduced math-like syntax
- Then came **C**, **Java**, **Python**, and many others
- Today's programmers cannot read code written 50 years ago

The problem: If the tools keep changing, what exactly are we learning?

The answer: There are mathematical properties of *problems themselves* — not of tools or hardware — that never change. TOC focuses on these timeless properties.

Two key benefits of this timeless theory:

1. It provides **abstract structures** useful for solving entire classes of problems. These can be implemented on any hardware or software platform available at any point in time.
2. It defines **provable limits** to what can ever be computed — regardless of processor speed or memory size. This prevents wasting effort on problems that are mathematically impossible to solve (like searching for a "perpetual motion machine" equivalent in computing).

Goal of TOC — the fundamental questions about any problem:

- Is there *any* computational solution to the problem? If not, is there a simpler version with a solution?
- If a solution exists, can it run with a *fixed amount of memory*?
- What is the *minimum time* needed to solve it?

---

ii. Applications of the Theory Are Everywhere

TOC is not just abstract mathematics — it appears in many real-world areas:

a) Communication Protocols & Languages Network protocols (like TCP/IP) are designed as formal languages. HTML describes web pages. Music can be modeled as a language. TOC provides the tools to design and verify these.

b) Programming Language Design & Compilers Every compiler (the program that translates your Python/Java code into machine instructions) uses **context-free grammars** for syntax definition and **parsing techniques** rooted in TOC.

> Example: When your compiler gives you a "syntax error," it has used a formal grammar to detect that your code violates a language rule.

c) Natural Language Processing (NLP) Programs that check grammar, power search engines, or perform machine translation all rely on context-free language theory.

d) Finite State Machines in Everyday Devices Systems like ATMs, vending machines, traffic lights, building security systems, and communication protocols are all modeled as **finite state machines** — one of the core topics in automata theory.

> Example: An ATM is a finite state machine. It transitions through states: *Idle* → *Card Inserted* → *PIN Entry* → *Transaction* → *Eject Card* → *Idle*. Each transition depends on the input (button pressed, PIN correct/incorrect, etc.).

e) Video Games Many interactive video games are large (often nondeterministic) finite state machines. Each enemy's behavior, each level's logic can be described this way.

f) Computational Biology DNA is literally a string of symbols (nucleotides: A, T, G, C). Proteins are strings of amino acids. Computational biologists use finite state machines and context-free grammars to analyze gene sequences — the same mathematical tools used for programming languages.

g) Security & Cryptography

- TOC's **undecidability results** prove that no general-purpose program can automatically verify all security properties of all programs.
- TOC's **complexity results** are the mathematical foundation for encryption algorithms (e.g., RSA, AES) that protect your online banking.

h) Artificial Intelligence Logical reasoning systems in AI (e.g., medical diagnosis programs) rely on formal logic frameworks. TOC proves that there is *no general theorem prover* that can automatically determine the truth of any arbitrary statement in first-order logic — a fundamental limit that AI researchers must work around.

---

3. Models of Computation: Purpose and Types

Why We Need Formal Models

Almost any statement about computation can be true in one model and false in another. This is why a **rigorous, formal definition** of a model of computation is essential — especially when trying to prove something is *impossible*.

> **Key Insight:** Proving that something *can* be computed is usually easy — you just write an algorithm. But proving that something *cannot* be computed requires first agreeing on what "computation" even means.

Concepts like "algorithm" and "computability" are intuitively understood, but intuition is not enough for mathematical proof. When we want to show that a problem has *no* solution, we must precisely define what tools are allowed.

Types of Models of Computation

a) Special-Purpose Models These models restrict the tools allowed to a specific set suited for a particular class of problems. They are efficient for their domain but cannot solve everything.

> Example: A **finite state machine** can check if a string of parentheses is balanced, but it *cannot* check if an arbitrarily nested expression is valid (that requires a pushdown automaton).

b) Universal Models These can simulate *any* computation — they are not limited to a specific class of problems. The study of universal models arose from the question: **"What can be computed by any algorithm, and what cannot?"**

This was studied in the **1930s** by:

- **Emil Post** (1897–1954)
- **Alonzo Church** (1903–1995) — developed the *lambda calculus*
- **Alan Turing** (1912–1954) — developed *Turing machines*

They each created different formal models — production systems, recursive functions, lambda calculus, Turing machines — and remarkably, **all these models turned out to be equivalent in power**. This equivalence strongly supports **Church's Thesis**.

> **Church's Thesis (Church-Turing Thesis):** Any function that is "intuitively computable" can be computed by a Turing machine. In other words, the Turing machine is a complete model of all possible computation.

Universal models have two essential ingredients:

1. **Unbounded memory** — no fixed limit on how much storage is available
2. **Unbounded time** — no fixed limit on how long a computation can run

> **Analogy:** A snail and a hare may differ in speed, but given unlimited time, the snail can cover the same ground as the hare. Similarly, a simple computing model can compute anything a complex one can — it just takes more steps and memory.

Two Examples of Universal Models

a) Markov Algorithms

- Access data and instructions *sequentially*
- Suited for computers with only sequential (tape-like) memory
- Uses string rewriting rules to perform computation

b) Random Access Machine (RAM)

- Based on *random access memory* — you can jump to any memory location instantly
- Follows the **von Neumann** architecture of modern computers (stored program design)
- This is essentially the model of your everyday laptop or smartphone

Characteristics of simulating a powerful machine on a simple one:

1. Straightforward *in principle*
2. Laborious and tedious *in practice*
3. Explodes (greatly increases) space and time requirements

> This is why the primitive operations of universal computing machines are kept as weak as possible — as long as the machine remains universal, the simplicity is desirable from a theoretical standpoint.

Computability vs. Complexity

The **theory of computability** (developed in the 1930s, expanded in the 1950s–60s) answers: *"Can this problem be solved at all?"*

However, computability theory only cares about *"computable in principle"* — it ignores whether a solution is actually feasible in practice. A program that takes 10^100 years to finish is "computable in principle" but completely useless.

Computational complexity theory (developed since the 1960s) fills this gap by classifying problems according to how much *time* and *space* they require. This is still an active area of research, including modern areas like:

- **Quantum computing** — using quantum mechanical properties to speed up computation
- **DNA computing** — using biological molecules to perform computation in parallel

---

4. Introduction to Formal Proof

Before diving into automata, we need a solid foundation in mathematical logic and proof techniques. These tools are used throughout the course.

Basic Symbols Used in TOC

Symbol	Meaning	Example
	Union	A B = elements in A or B or both
	Intersection (Conjunction)	A B = elements in both A and B
(or)	Empty String	A string with no characters
Ø (Φ)	Null / Empty Set	A set with no elements
¬ (7)	Negation	¬p = "not p"
A'	Complement	Everything NOT in A (within universal set U)
	Implies	p q means "if p then q"

Important Laws:

- **Additive inverse:** a + (-a) = 0
- **Multiplicative inverse:** a × (1/a) = 1
- **Universal set example:** U = {1, 2, 3, 4, 5}, A = {1, 3}, so A' = {2, 4, 5}

Absorption Laws:

- A (A B) = A
- A (A B) = A

  Absorption explained: If A = {1,2} and B = {2,3}, then A B = {2}, and A {2} = {1,2} = A. The larger set "absorbs" the union or intersection with a subset.

De Morgan's Laws:

- (A B)' = A' B'
- (A B)' = A' B'

  De Morgan's in plain English: "Not (A or B)" is the same as "(Not A) and (Not B)." Like saying "I don't want tea or coffee" means "I don't want tea AND I don't want coffee."

Double Complement:

- (A')' = A *(Taking the complement twice gives you back the original set)*

A A' = Ø *(A set and its complement share no elements)*

Relations

Let A and B be two sets. A **relation** R is a subset of A × B (the Cartesian product — all possible pairings).

Key types of relations used in TOC:

Relation Type	Definition	Example
Reflexive	Every element relates to itself: aRa for all a	"=" (equals): 5 = 5
Symmetric	If aRb then bRa	"is a sibling of": if A is sibling of B, then B is sibling of A
Transitive	If aRb and bRc then aRc	"is less than": if 2 < 4 and 4 < 7, then 2 < 7

Equivalence Relation: A relation that is *reflexive*, *symmetric*, and *transitive* simultaneously.

> **Example of an equivalence relation:** "=" (equality on numbers) — 5=5 (reflexive), if 5=5 then 5=5 (symmetric), if a=b and b=c then a=c (transitive).
>
> **Non-example:** " " (less than or equal) is reflexive and transitive, but not symmetric (5 7 does not mean 7 5), so it is NOT an equivalence relation.

---

5. Proof Techniques

a) Deductive Proof

A **deductive proof** is a sequence of logical statements that leads from a **hypothesis** (given facts) to a **conclusion**, where each step is justified by rules of logic or previously established facts.

The theorem proved has the form: **"If H then C"** — we say C is *deduced* from H.

---

b) Additional Forms of Proof

- Proof of sets (showing set membership or equality)
- Proof by contradiction
- Proof by counter-example

---

c) Direct Proof (also called Constructive Proof)

Form: If P is true, then Q is true — proved directly by constructing the logical chain.

Example: *If a and b are odd numbers, then their product is also odd.*

Proof:

- Represent any odd number as 2n+1
- Let a = 2x + 1 and b = 2y + 1
- Then: a × b = (2x + 1)(2y + 1) = 4xy + 2x + 2y + 1 = 2(2xy + x + y) + 1 = 2z + 1 (which is odd)

  The product of any two odd numbers is always odd — this has been directly constructed from the definition.

---

d) Proof by Contrapositive

The **contrapositive** of the statement *"If H then C"* is *"If not C, then not H."*

A statement and its contrapositive are **logically equivalent** — both are true or both are false. It is sometimes easier to prove the contrapositive than the original.

Why they are equivalent — four cases to consider:

H	C	"If H then C"	"If not C, then not H"
True	True	True	True
True	False	**False**	**False**
False	True	True	True
False	False	True	True

In all four cases, both statements have the same truth value — they are logically equivalent.

Example — Theorem 1.10: R (S T) = (R S) (R T)

This can be proved in two parts (if and only if):

"If" direction: Prove that if x R (S T), then x (R S) (R T)

Step	Statement	Justification
1	x R (S T)	Given
2	x R or x S T	Definition of union
3	x R or x both S and T	Definition of intersection
4	x R S	Definition of union
5	x R T	Definition of union
6	x (R S) (R T)	Steps 4, 5, and definition of intersection

"Only-if" direction: Prove the reverse (if x (R S) (R T), then x R (S T)) follows by reversing the logic steps.

---

e) Proof by Contradiction

Form: Assume both the hypothesis H *and* the negation of the conclusion (not C). Show that this leads to a logical **falsehood** (something we know is impossible). Therefore, the original statement must be true.

Formal example:

For any sets a, b, c: if a b = Ø and c b, prove that a c = Ø

Given: a b = Ø and c b

Proof:

- **Assume** (for contradiction) that a c Ø
- Then there exists some x such that x a and x c
- Since c b, x c implies x b
- Therefore x a and x b, which means a b Ø
- But this **contradicts** our given fact that a b = Ø
- Therefore our assumption was wrong, and a c = Ø

Alleged Theorem 1.13 (disproof by counter-example): "All primes are odd." **Disproof:** The integer 2 is prime, but 2 is even. This single counter-example is sufficient to disprove the entire claim.

---

f) Proof by Mathematical Induction

Used to prove statements of the form S(n) that must hold for all integers n some starting value.

Two steps:

1. **Basis (Base Case):** Prove S(i) for a specific starting integer i (usually i = 0 or i = 1).

2. **Inductive Step:** Assume S(n) is true for some n i (the *inductive hypothesis*), then prove S(n+1) must also be true.

The Induction Principle:
If we prove S(i) [base case] and we prove that S(n) S(n+1) [inductive step], then we may conclude S(n) is true for **all n i**.

> **Analogy:** Mathematical induction is like an infinite row of dominoes. If you knock over the first one (base case), and each domino knocks over the next one (inductive step), then *all* dominoes will fall.
>
> **Example:** Prove that 1 + 2 + 3 + … + n = n(n+1)/2
>
> - **Base case (n=1):** LHS = 1, RHS = 1(2)/2 = 1
> - **Inductive step:** Assume true for n. For n+1: 1+2+…+n+(n+1) = n(n+1)/2 + (n+1) = (n+1)(n+2)/2

---

6. Languages

What is a Formal Language?

In TOC, a **language** is an abstraction of natural languages (like English), but with precise, formal definitions. Programming languages belong to this category. We focus on formal languages because:

- Natural languages are ambiguous ("I saw the man with a telescope" — who has the telescope?)
- Formal languages have exact, unambiguous rules — essential for computers

---

Symbols

A **symbol** is the most basic, indivisible unit — it cannot be broken down further. Symbols are the *atoms* of the language world.

> Examples of symbols: `a`, `0`, `1`, `#`, `begin`, `do`, `$`

Alphabets (Σ)

An **alphabet** Σ (sigma) is a **finite, non-empty set of symbols**.

> Examples:
>
> - Σ = {0, 1} — binary alphabet (used in computers)
> - Σ = {a, b, c} — small letter alphabet

- Σ = {a, b, c, &, z} — mixed alphabet
- Σ = {#, , , } — symbolic alphabet

When multiple alphabets are used, subscripts distinguish them: Σ , Σ , etc.

Strings (Words) over an Alphabet

A **string** (or **word**) over alphabet Σ is a finite sequence of concatenated symbols from Σ.

Examples:

- `0110`, `11`, `001` are strings over Σ = {0, 1}
- `aab`, `abcb`, `b`, `cc` are strings over Σ = {a, b, c}

Important note: A string does NOT have to use all symbols from the alphabet.

Example: `cc` is a valid string over {a, b, c} even though it doesn't contain `a` or `b`.

Consequence: A string over Σ is also a valid string over any *superset* of Σ. So `cc` is valid over {a, b, c}, over {a, b, c, d}, over {a, b, c, #, 1}, etc.

The Empty String ( )

The **empty string** (epsilon) is the string with **zero symbols**. It belongs to every alphabet's set of strings.

Think of it as an "empty word" — like the empty line before you start writing.

Length of a String

The **length** of a string w, written |w|, is the number of symbols it contains.

Examples:

- |011| = 3
- |11| = 2
- |b| = 1
- | | = 0

String Operations

Concatenation The **concatenation** of strings x and y, written **xy**, is the string x followed immediately by y with no space in between.

Example: If x = `ab` and y = `cd`, then xy = `abcd`

Prefixes, Suffixes, and Substrings For the string `011`:

Type	Values	Explanation
Prefixes	, 0, 01, 011	Strings that begin the word (including and the whole string)
Suffixes	, 1, 11, 011	Strings that end the word (including and the whole string)
Substrings	, 0, 1, 01, 11, 011	Any contiguous portion of the string

Key rules:

- is always a prefix, suffix, and substring of any string
- Any string x is a prefix, suffix, and substring of itself
- A **proper prefix (suffix)** of y is a prefix (suffix) that is NOT equal to y itself

  In the example above, the proper prefixes of `011` are: , 0, 01 (everything except `011` itself)

Summary Table: Key Concepts

Concept	Definition	Example
Symbol	Indivisible unit of a language	`a`, `0`, `#`
Alphabet (Σ)	Finite non-empty set of symbols	{0, 1}, {a, b, c}
String	Finite sequence of symbols	`0110`, `abc`
Empty string ( )	String of length 0	
Length (|w|)	Number of symbols in string w	|011| = 3
Language	Set of strings over an alphabet	{w : w contains equal 0s and 1s}

Concept	Definition	Example
Concatenation (xy)	x followed by y	if x=`ab`, y=`cd` then xy=`abcd`
Prefix	Beginning portion of a string	Prefixes of `011`: , 0, 01, 011
Suffix	Ending portion of a string	Suffixes of `011`: , 1, 11, 011
Substring	Any contiguous portion	Substrings of `011`: , 0, 1, 01, 11, 011

These notes are based on the CSC 452 course material. Study tip: For each proof technique, practice at least two or three examples of your own — understanding the structure is more valuable than memorizing the steps.